

CmpE 485 - Term Project

Xonix3D

Mustafa Batuhan Büber

2021400231

GitHub Repository

github.com/mubatu/Xonix3D

Contents

1	Motivation	2
2	Design	2
3	Technical Challenges	3
3.1	Path Tiles	3
3.2	Area Capture	4
3.3	Ball Movement	6
4	Performance Evaluation	8
5	Demo Video	8
6	Conclusion	8

1 Motivation

Xonix is one of the well-known classic games. It was released in 1984. Its graphics are not fancy, but the gameplay is very fun and competitive.

I really enjoyed playing Xonix. Because of this, I wanted to make a modern 3D version of the game. My goal was to keep the main gameplay idea of Xonix and present it in a 3D environment.

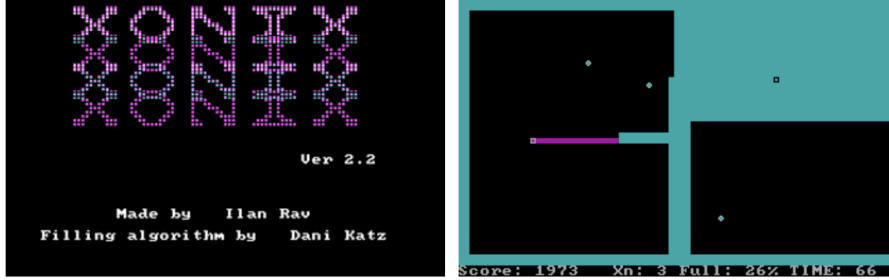


Figure 1: The original Xonix game.

2 Design

Xonix3D is a level-based game. In each level, the player tries to capture enough area to pass the level. The player should do this without touching the balls.

The player can see the progress bar, remaining time, and lives at the top of the screen. The progress bar shows how much area is captured. The remaining time shows how much time is left for the level. The lives show how many more chances the player has.

There are two different types of balls in the game. The first type only rolls inside the empty area. The second type can also eat the captured areas. This makes the game harder because the player can lose some of the captured area.

When a ball hits the player, the player dies. The player also dies when a ball hits the temporary path. In this case, the path starts to burn. If the burning path reaches the player before the path is completed, the player loses one life.



Figure 2: A gameplay scene from Xonix3D.

3 Technical Challenges

3.1 Path Tiles

When the player creates a path, orange tiles appear on the grid. These tiles show the temporary path of the player.

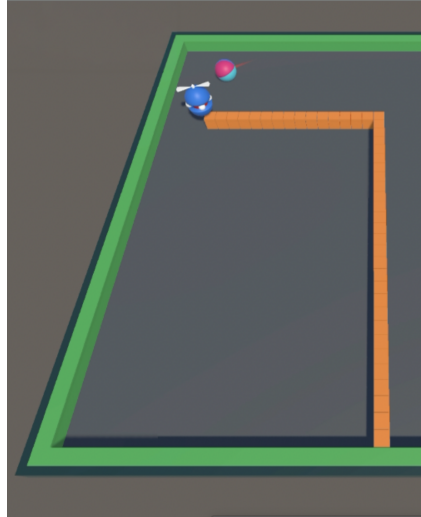


Figure 3: Orange tiles created by the player path.

At first, I created and destroyed these tiles every time. This put unnecessary load on the system. Creating many objects during gameplay can increase CPU usage and cause performance problems.

The solution was object pooling. Instead of destroying a tile, I keep it in a pool. When I need a new path tile, I take one from the pool. This reduces the number of new object creations during gameplay.

The profiler images show this difference. The first image shows the spikes that happen while creating a path tile. This was taken from the Profiler when I created the first path after starting the game.

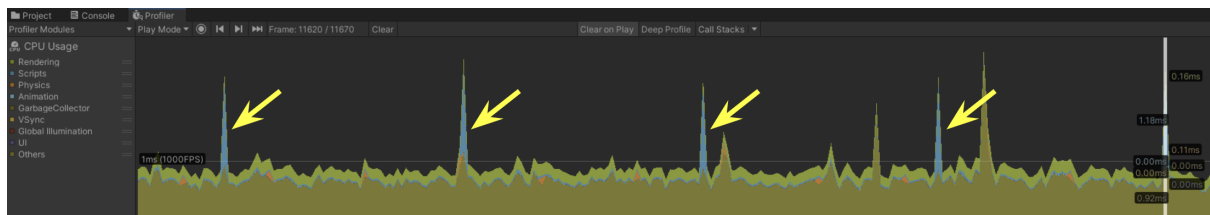


Figure 4: CPU spikes while creating path tiles.

The next two images compare the first run and the second run. The first one shows the first time I created the path. The second one shows when I created the same path for the second time. We can clearly see the drop in CPU usage.

Overview	Total	GC Alloc	Time ms
Xonix.PathTiles.AddTemporaryPathCell	3.7%	268 B	0.09
Xonix.PathTiles.RefreshCellVisual	3.6%	268 B	0.09
Xonix.PathTiles.ShowOrReuse	3.6%	268 B	0.09
Xonix.PathTiles.CreateRenderer	3.3%	268 B	0.08

Figure 5: Profiler result for the first path creation.

Overview	Total	GC Alloc	Time ms
Xonix.PathTiles.AddTemporaryPathCell	0.9%	0 B	0.02
Xonix.PathTiles.RefreshCellVisual	0.8%	0 B	0.01
Xonix.PathTiles.ShowOrReuse	0.8%	0 B	0.01

Figure 6: Profiler result for the second path creation.

The next plot shows CPU time by path length. As the path length gets bigger, we can easily see that the difference between the cold run and the warm run also gets bigger. There is about an 82 percent decrease in CPU times.

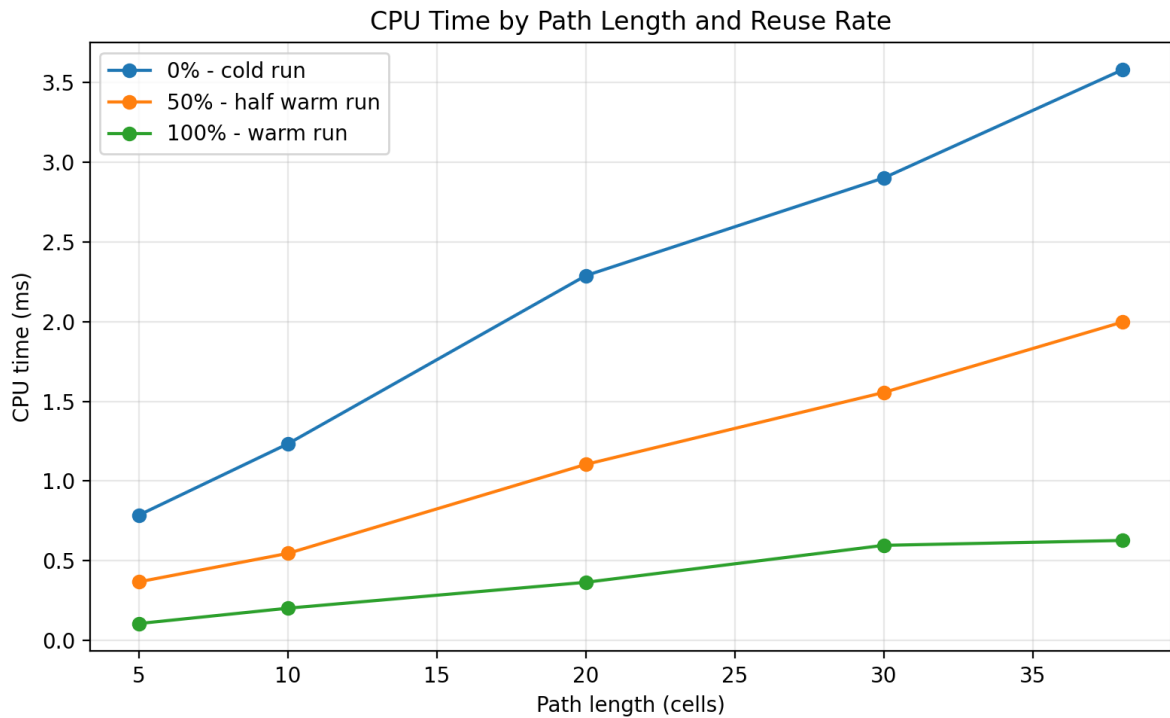


Figure 7: CPU time by path length.

3.2 Area Capture

When the player completes a path, that area becomes captured. The captured area is shown with different tiles on the grid.

While visualizing this, rendering each cell one by one results in a performance drop. This is a problem because a large captured area can contain many cells.

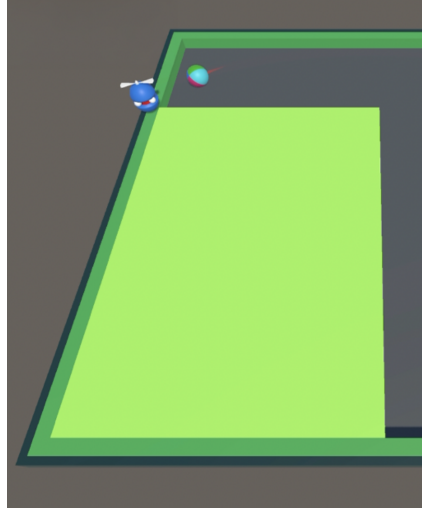


Figure 8: Area capture after completing a path.

In the optimized version, the captured cell states and visuals are updated all at once with a bulk grid update. This reduces the total work during area capture.

We can clearly see the drop in CPU usage from these profiler results.

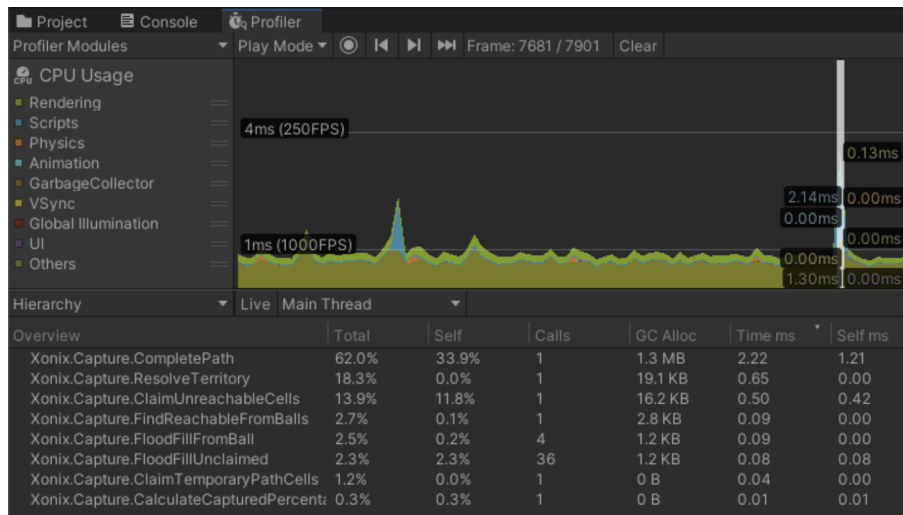


Figure 9: Profiler result for the naive area capture update.

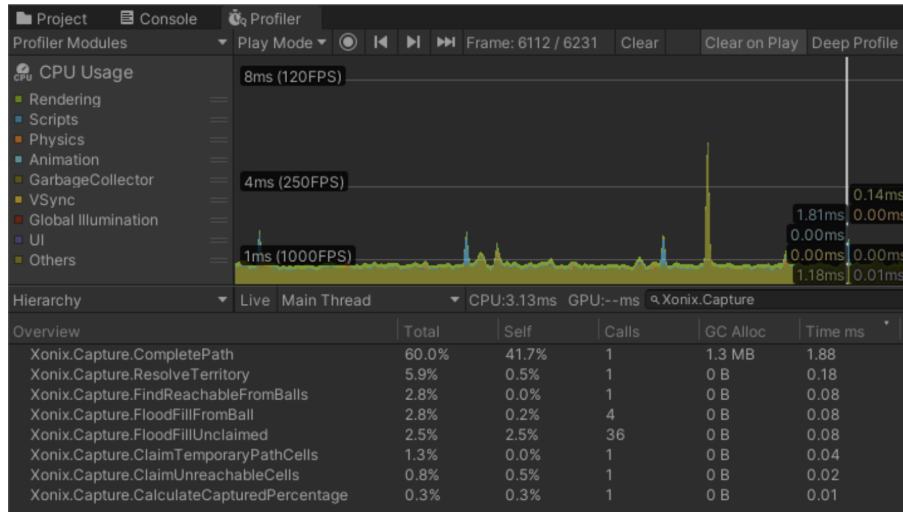


Figure 10: Profiler result for the optimized area capture update.

The next graph shows complete path time by grid size. Here, we can see the difference between the naive solution and the optimized solution for different grid sizes.

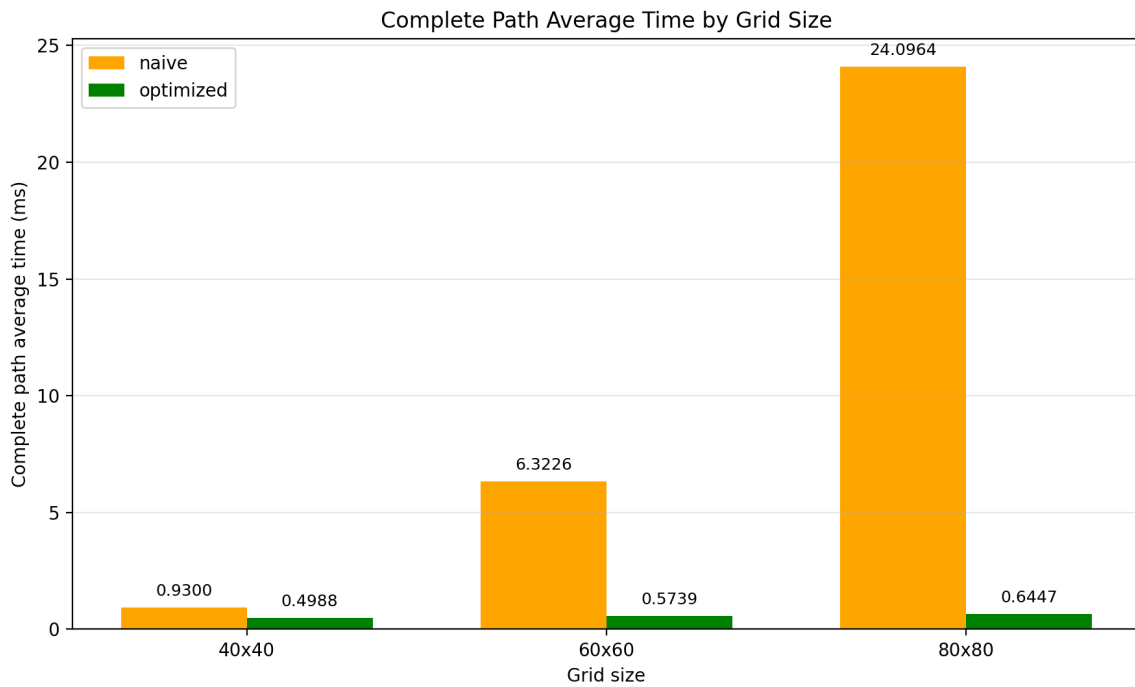


Figure 11: Complete path time by grid size.

3.3 Ball Movement

At first, I did not use Unity ball physics. The game is grid-based, and I thought that using thousands of grid colliders would be expensive.

To handle ball movement without Unity Physics, I divided the movement into smaller steps. Otherwise, tunneling or missed collisions could happen. When a collision was

detected, the system used the last safe position of the ball. Then it reflected the ball direction.

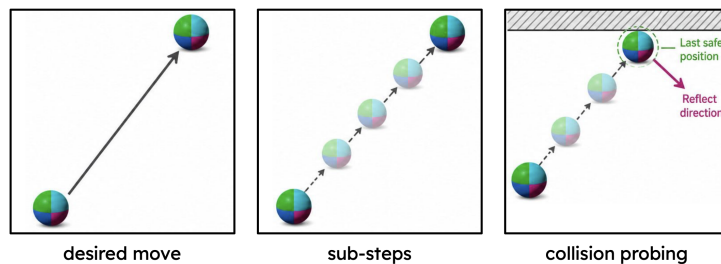


Figure 12: Sub-stepped probing technique for ball movement.

Later, I tried Unity PhysX after Atay Hoca suggested it. I first expected it to cause performance problems, but the result was surprising. The pure physics version was faster than my custom movement system, and also made the ball collisions more realistic.

After that, I made one more improvement for the colliders. Instead of using one collider for each blocked cell, the grid uses fewer and larger colliders. Horizontal blocked cells are merged into larger BoxColliders. This made the best performance result.

PhysX is fast in this case because it has optimized systems for collision detection and contact solving. It also uses native C++ data structures. Even though real physics sounds heavier, it can be faster than repeated collision checks written in C#.

In the final version, the balls do not collide with thousands of small cell colliders. They collide with fewer and larger static colliders. This is a good case for PhysX.

The new BallController also does less custom work. It mostly sets and keeps the velocity, handles gameplay effects, and lets PhysX choose the bounce direction.

The next plot shows the FPS difference between the old sub-stepped solution and the latest PhysX solution.

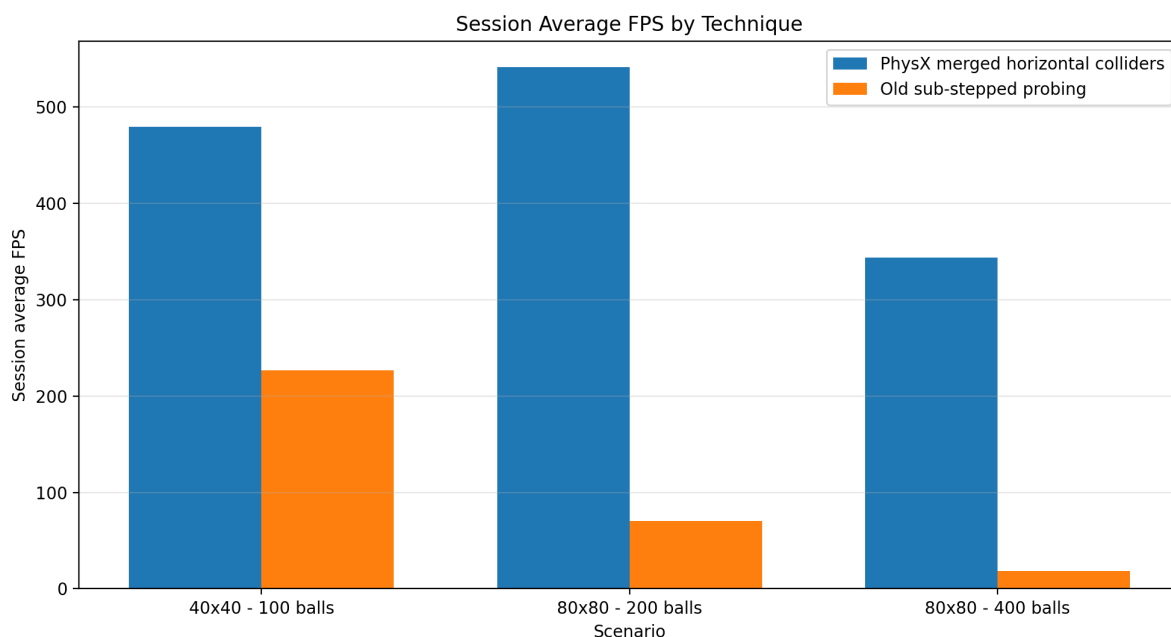


Figure 13: Average FPS by ball movement technique.

4 Performance Evaluation

To measure the game performance, I used grid size and ball count as parameters. The Y-axis shows the average FPS.

Since I do not expect to use more than 100 balls in the levels, the performance seems good enough. For this range, the FPS is higher than 200.

Also, from this graph, I understood that grid size does not affect FPS much. However, the ball count has a big effect on FPS.

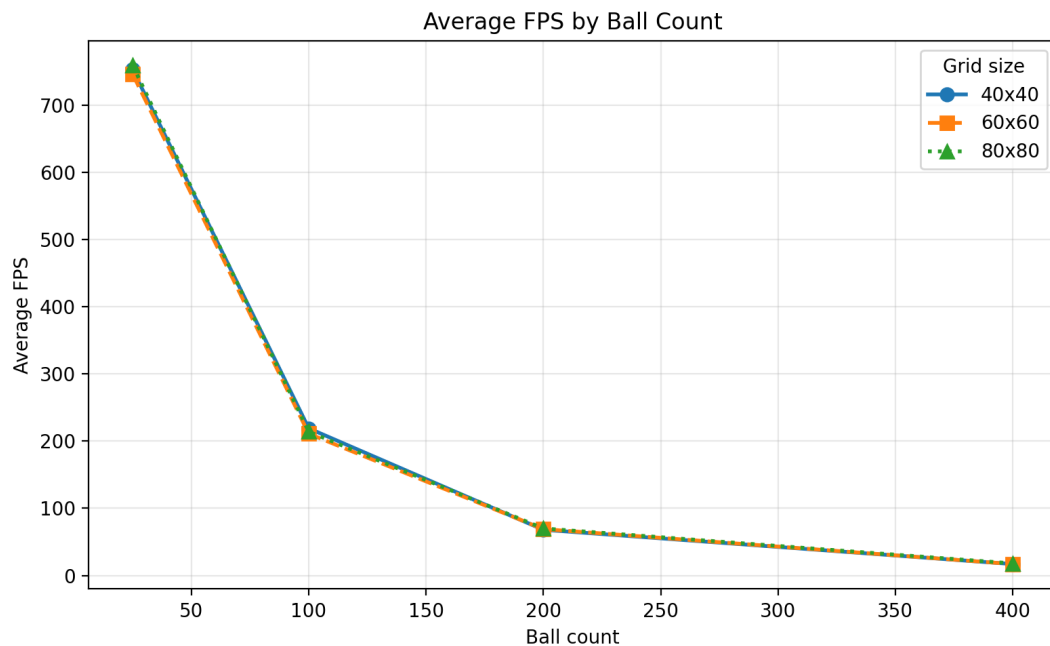


Figure 14: Average FPS by ball count and grid size.

5 Demo Video

30 second demo gameplay video link is [here](#).

6 Conclusion

During the development process, I learned how to use the Profiler in a real game project. I could see which parts of the game were expensive, and I could test my ideas with real data. This helped me understand the effect of optimization much better.

I also saw that the first solution is not always the best solution. For example, I first tried to handle ball movement with my own code. Later, I changed it to Unity PhysX and got better results. This showed me that testing different approaches is very important in game development.

In the end, I am happy with the result. I was able to take a classic game that I enjoy and build a 3D version of it. Now I can play Xonix in a new way, with my own design and implementation.